MIRRORWORLD

A Constraint-Aware Campus Digital Twin for Counterfactual Simulation and Sustainable Resource Optimization

Project Documentation for BCS ICT Fest 2026 – Module 12: Innovation Showcase / Exhibition

Judges' Quick View

> **In one sentence:** MIRRORWORLD lets a university **test a decision on a digital copy of its campus first**, then shows the predicted results, the best alternative plans and their trade-offs before anything changes in real life.

What judges look for	How MIRRORWORLD answers it	Section
Novelty	**Counterfactual simulation + dependency analysis + multi-objective optimization + explainable trade-offs** in one workflow	5
Practicality	**A real, recurring problem:** more students, limited rooms, lab and teacher constraints	3, 4
Impact	**Measured against a baseline:** conflicts, capacity violations, utilization, energy (kWh/week)	11, 17
Implementation	**Working engines:** OR-Tools CP-SAT optimizer, simulation engine, FastAPI backend, PostgreSQL, React frontend	7–14
Presentation	**Live 5-minute demo:** judges change a condition and watch the results recalculate	21
SDG alignment	**SDG 9, 11 and 12** with measurable indicators	18

> **Honesty note:** every result value is calculated by the software, and estimated or simulated values are labeled as such.

1. Project Summary

Item	Detail
Project title	MIRRORWORLD
Tagline	Test the future of your campus before you change it.
Project type	Software-based (web application)
Category	Digital twin + simulation + optimization
Primary SDGs	SDG 9 (Industry, Innovation and Infrastructure), SDG 11 (Sustainable Cities and Communities)
Supporting SDG	SDG 12 (Responsible Consumption and Production)
Target users	University administrators, registrar and timetable offices, department heads
Core technologies	React, TypeScript, Python, FastAPI, PostgreSQL, Google OR-Tools (CP-SAT), SimPy
Safety	Fully software-based; no hardware or hazardous demonstration

2. Abstract

Universities make resource decisions, such as adding students or moving laboratories, **without a way to test the consequences first**. **MIRRORWORLD is a campus digital twin** that models rooms, labs, teachers and student groups with real operational constraints. Administrators create what-if scenarios (for example, +200 students or a lab closure), and the platform **simulates the impact, generates alternative timetables with an OR-Tools optimizer, and compares them with the baseline** on conflicts, room utilization, energy use and teacher workload, then explains the trade-offs. It supports **SDG 9, 11 and 12**.

3. Problem Statement

University timetabling and resource planning are tightly coupled problems. Moving one laboratory session can affect teacher availability, room availability, student schedules, equipment use, congestion and electricity consumption at the same time. In practice, administrators often rely on manual calculation, experience or trial and error, and discover problems only after the change is implemented.

Specific gaps:

- Management systems store records but do not let users experiment with future states.
- Timetable generators produce a schedule but rarely explain wider consequences or trade-offs.
- There is little support for comparing several alternatives against a measured baseline.
- Sudden disruptions (room closure, teacher absence, enrollment growth) are handled manually.
- Energy and utilization impact is rarely considered during scheduling.

4. Proposed Solution and Objectives

Solution. A configurable digital model of a university campus, combined with a simulation engine, a constraint-based optimizer, a dependency-impact analyzer and an explainable comparison interface.

Objectives

1. Build a structured digital model of campus resources and operational rules.
2. Let users create counterfactual scenarios without altering the baseline.
3. Generate valid alternative timetables through constraint optimization.
4. Quantify outcomes with transparent mathematical metrics.
5. Explain trade-offs and unresolved constraints in plain language.
6. Support disruption recovery when a resource becomes unavailable.
7. Record every experiment for reproducibility and audit.

The five guiding questions: What is happening now? What happens if we change X? Which constraints or resources become problematic? Which alternative performs best? Why, and at what cost?

5. Novelty and Differentiation

Digital twins and timetable optimization are established fields, so MIRRORWORLD does not claim to invent them. **Its novelty is the integrated combination of four capabilities in one decision workflow:**

1. **Counterfactual simulation:** change one condition, keep the rest constant, and measure the effect against the baseline.
2. **Dependency-aware impact analysis:** a resource graph identifies exactly which teachers, courses, groups and rooms are affected by a change.
3. **Multi-objective optimization:** several plans (utilization-first, energy-first, balanced) are generated and compared, with a Pareto view of trade-offs.
4. **Explainable recommendations:** explanations are generated from computed results, not from a chatbot.

Before final submission, review existing campus scheduling and digital-twin tools and state precisely how this implementation differs.

6. Features

6.1 Core features (must be complete)

#	Feature	Description
1	Digital Campus Model	CRUD for buildings, rooms, labs, equipment, teachers, courses, student groups and timetable; seeded sample campus dataset
2	What-If Scenario Builder	Change student count, room availability, lab policy, operating hours, energy mode; baseline stays untouched
3	Simulation Engine	Computes conflicts, utilization, capacity violations, workload, movement and energy for a scenario
4	Optimization Engine	OR-Tools CP-SAT assigns classes to rooms and slots under hard and soft constraints
5	Before/After Comparison	Baseline vs. scenarios side by side with charts and a timetable view

#	Feature	Description
6	Explainable Recommendations	Rule-based explanation generated from metric differences and constraint results

6.2 Advanced features

#	Feature	Description
7	Multi-Scenario Decision Lab	Generates Plans A–D (existing, max utilization, energy-first, balanced) and a Pareto trade-off chart
8	Resource Dependency Graph	Graph traversal shows affected sessions, teachers, groups and alternative rooms
9	Disruption and Recovery	Mark a lab, room or teacher unavailable; system finds a revised plan and lists unresolved issues
10	Sustainability Intelligence	Transparent energy model with documented assumptions
11	Decision History / Audit Trail	Every run stored with parameters, objectives, results, warnings and timestamp

6.3 Optional premium features (only if the core is finished)

- AI Decision Copilot: converts a typed request into a structured scenario and calls the real engine; it never invents numbers.
- Sensitivity analysis: low/expected/high energy assumptions to test whether the recommendation is robust.
- Interactive 2D campus map showing room status after each simulation.
- PDF/CSV export of scenario reports.

7. System Architecture

7.1 Layered architecture

Layer	Responsibility	Technology
Presentation	Dashboard, editor, scenario builder, comparison, graph and map views	React, TypeScript, Vite
API	REST endpoints, validation, authentication, job orchestration	FastAPI, Pydantic
Domain / Services	Scenario service, metrics service, explanation service, graph service	Python
Engines	Simulation engine and optimization engine	SimPy, OR-Tools CP-SAT, NetworkX
Data	Campus model, scenarios, results, audit log	PostgreSQL, SQLAlchemy, Alembic
Deployment	Containerized services	Docker Compose

7.2 Request flow (one experiment)

1. Frontend sends a scenario definition (parameter changes) to the API.
2. API validates input with Pydantic schemas and checks data consistency.
3. Scenario service loads the baseline from PostgreSQL and creates a deep copy with the changes applied.
4. Simulation engine evaluates the scenario and computes indicators.
5. Optimization engine runs with a time limit and returns one or more candidate timetables, or reports infeasibility.
6. Metrics service computes all indicators for each candidate and the baseline.
7. Explanation service compares results and produces structured explanations.
8. Results, assumptions and warnings are stored in the audit log and returned to the UI.

Long-running optimization runs are executed as background jobs (FastAPI background tasks or a lightweight worker such as RQ/Celery with Redis). The frontend polls a status endpoint and shows progress.

7.3 Key design principles

- Baseline is immutable; every experiment is a separate scenario record.
- No fabricated numbers: every displayed value comes from the engine and is traceable.
- Data is labeled as Observed, Estimated, Simulated or Optimized.
- Hard constraints are enforced in the solver, not penalized.
- Deterministic runs: the same input and random seed produce the same output.

8. Backend Design

8.1 Folder structure

- app/main.py: FastAPI entry point
- app/api/: routers (campus, scenarios, simulation, optimization, comparison, history)
- app/schemas/: Pydantic request/response models
- app/models/: SQLAlchemy ORM models
- app/services/: scenario, metrics, explanation, graph, export
- app/engines/simulation/: simulator, energy model, movement model
- app/engines/optimizer/: CP-SAT model builder, constraints, objectives, solver runner
- app/core/: configuration, security, logging
- tests/: unit and integration tests
- seed/: sample campus generator

8.2 REST API (main endpoints)

Method	Endpoint	Purpose
GET/POST	/api/campus/{buildings, rooms, labs, teachers, courses, groups}	Manage campus model
GET	/api/timetable/baseline	Retrieve baseline timetable
POST	/api/scenarios	Create a scenario from parameter changes
GET	/api/scenarios/{id}	Scenario details and status
POST	/api/scenarios/{id}/simulate	Run simulation

Method	Endpoint	Purpose
POST	/api/scenarios/{id}/optimize	Generate candidate plans (objective weights, time limit)
POST	/api/scenarios/{id}/disrupt	Apply disruption and run recovery
GET	/api/compare?ids=...	Compare baseline and scenarios
GET	/api/graph/impact?resource=...	Dependency impact analysis
GET	/api/history	Audit trail of experiments
GET	/api/scenarios/{id}/export	Export report (CSV/PDF)

8.3 Security and quality

- Role-based access (Admin, Viewer) with JWT authentication.
- Input validation, parameterized queries, CORS configuration, rate limiting on solver endpoints.
- Solver time limits to keep the system responsive.
- Automated tests for constraints, metrics and API (pytest); sample-data fixtures.

9. Frontend Design

9.1 Pages

Page	Content
Dashboard	Baseline KPIs: utilization, conflicts, energy estimate, teacher workload
Campus Model Editor	Tables and forms for rooms, labs, teachers, courses, groups
Timetable View	Weekly grid by room, teacher or group; conflict highlighting
Scenario Builder	Parameter controls (students, rooms, hours, lab policy, energy mode) and Run Simulation button

Page	Content
Decision Lab	Candidate plans A–D, objective weight sliders, Find Better Plans button, Pareto chart
Comparison	Baseline vs. plans: metric table, bar/radar charts, timetable diff
Impact Graph	Interactive dependency graph for disruption analysis
Disruption Center	Mark resource unavailable, run recovery, view unresolved items
Campus Map (optional)	2D building/room status view
History	Past experiments, rerun and compare

9.2 Frontend stack

- React + TypeScript with Vite; React Router for navigation.
- TanStack Query for API state and polling; Zustand for local UI state.
- Tailwind CSS with a component library such as shadcn/ui.
- Recharts for charts; React Flow for the dependency graph; a table component for timetables.
- Clear labels distinguishing baseline, estimated and simulated values; warnings for infeasible scenarios.
- Responsive layout suitable for projector and laptop demonstration.

10. Database Design

Table	Key fields
buildings	id, name, operating_start, operating_end
rooms	id, building_id, name, type (classroom/lab), capacity, power_kw, equipment tags
equipment	id, name, room_id, quantity, power_kw
teachers	id, name, max_weekly_hours, availability (slots)

Table	Key fields
courses	id, code, teacher_id, weekly_sessions, session_length, needs_lab, required_equipment
student_groups	id, department, batch, size
group_courses	group_id, course_id
time_slots	id, day, start, end
timetable_entries	id, version_id, course_id, room_id, slot_id, group_id
scenarios	id, name, baseline_version, parameters (JSONB), created_at
scenario_results	id, scenario_id, plan_label, metrics (JSONB), timetable (JSONB), warnings, assumptions, solver_status
audit_log	id, user_id, action, scenario_id, timestamp

11. Mathematical Model

11.1 Room utilization

U = (Occupied room-hours / Available room-hours) × 100%

Example: 30 occupied hours out of 40 available gives U = 75%. High utilization is not automatically good; some slack is needed for flexibility.

11.2 Capacity violation

V_i = max(0, S_i − K_i), where S_i is the number of students in class i and K_i is the capacity of the assigned room. Total violation = sum of V_i. Valid plans require this to be 0.

11.3 Scheduling conflicts

A conflict is counted when two sessions that share a teacher, room or student group occupy the same time slot. In the optimizer these are forbidden by hard constraints; in the baseline and in disrupted scenarios they are counted as diagnostics.

11.4 Energy estimate

E = sum over rooms r of (P_r × H_r), where P_r is the modeled power load in kW and H_r the operating hours. A refined model separates lighting, air conditioning, computers and laboratory equipment. Assumptions are documented and kept identical across scenarios. The model does not capture weather or real air-conditioning behavior, and this is stated as a limitation.

11.5 Teacher workload balance

W = standard deviation of weekly teaching hours across teachers (lower is more balanced), also reporting maximum load against each teacher's limit.

11.6 Student movement

M = sum over consecutive session pairs of a group of d(building_a, building_b), where d is a building-distance (or transition-penalty) matrix.

11.7 Objective function

Minimize Z = w1·C + w2·E + w3·M + w4·W + w5·G, where C is soft-constraint violations, E is normalized energy, M is movement, W is workload imbalance, G is schedule gaps, and w are user-adjustable weights. Each term is normalized before weighting so the weights are comparable.

12. Optimization Engine (OR-Tools CP-SAT)

Decision variables: x[s, r, t] ∈ {0,1} equals 1 if session s is placed in room r at time slot t.

Hard constraints

1. Every session is assigned exactly once (or reported as unassigned in a relaxed run).
2. A room hosts at most one session per slot.
3. A teacher teaches at most one session per slot and only in available slots.
4. A student group attends at most one session per slot.
5. Group size must not exceed room capacity.
6. Required equipment and lab type must match the room.
7. Rooms and slots marked unavailable are excluded.
8. Teacher weekly hours must not exceed the maximum.

9. Multi-slot sessions (labs) occupy consecutive slots.

Soft constraints (penalized in the objective): gaps between classes, long inter-building movement, workload imbalance, peak-hour congestion, energy-inefficient room choices, preferred lab time windows.

Behavior: a time limit is set for each solve; the solver status (OPTIMAL, FEASIBLE, INFEASIBLE) is reported honestly. If infeasible, the system explains which constraint groups conflict using relaxed re-runs. Plans A–D are produced by re-solving with different weight sets.

13. Simulation Engine

- **Static evaluation:** computes all metrics for any given timetable and campus configuration.
- **Counterfactual run:** clones the baseline, applies parameter changes (for example, +200 students split proportionally across groups), and re-evaluates.
- **Discrete-event simulation (SimPy):** models student movement between sessions, corridor or room queues and peak congestion within a day, using a documented process model.
- **Disruption module:** removes a resource, uses the dependency graph to find affected sessions, tries local repair first, then a full re-optimization, and reports unresolved items.
- **Reproducibility:** fixed random seeds and stored parameters.

14. Dependency Graph and Impact Analysis

Nodes: teachers, courses, sessions, student groups, rooms, buildings, equipment. Edges: teaches, attends, assigned-to, located-in, requires. Implemented with NetworkX; traversal (BFS) from a disrupted node returns affected sessions, teachers and groups, and candidate alternative rooms and slots. Results are rendered in React Flow.

15. Explanation Engine

Rule-based templates fed by computed metric differences. Example logic: if conflicts fall and afternoon utilization rises, state that conflicts were reduced by spreading laboratory sessions across more slots at the cost of less afternoon flexibility. Each statement

references the metric and value that triggered it. An optional LLM layer may rephrase the structured result but must never create numbers.

16. Demonstration Dataset and Baseline

- Synthetic but realistic campus: about 1,000 students, 20 classrooms, 4 laboratories, 30–40 teachers, 25–30 student groups, a weekly timetable of roughly 100 sessions.
- Baseline deliberately contains a few real conflicts and overloaded rooms so improvements are measurable.
- Parameters (capacities, power ratings, distances) documented in an assumptions table.
- If real university data is later available, import via CSV.

17. Evaluation Plan and Metrics

Test scenarios: S1 +200 students; S2 three classrooms unavailable; S3 all labs moved to afternoon; S4 one laboratory closed; S5 energy-saving priority.

For each scenario report: scheduling conflicts, unassigned classes, capacity violations, average and peak room utilization, estimated energy (kWh/week), teacher workload deviation, student movement, solver status and runtime.

Important: fill in the results table below only with values your software actually produced.

Metric	Baseline	S1 Plan A	S1 Plan D
Conflicts	[measured]	[measured]	[measured]
Capacity violations	[measured]	[measured]	[measured]
Avg room utilization	[measured]	[measured]	[measured]
Energy (kWh/week, estimated)	[measured]	[measured]	[measured]
Solver runtime (s)	–	[measured]	[measured]

Testing: unit tests for each constraint, tests that known-infeasible inputs return INFEASIBLE, and cross-checks of metrics against manual calculations on a small dataset.

18. SDG Alignment

SDG	Contribution	Indicator measured
9	Digital modeling and optimization to make better use of existing infrastructure; evaluate infrastructure decisions before building	Room and lab utilization, capacity violations avoided
11	More efficient, resilient institutional operations in the built environment; congestion reduction	Peak congestion, student movement, recovery success rate
12	Reduce avoidable resource use through energy-aware scheduling	Estimated kWh per week under each plan

Claims are limited to what the model measures; energy results are estimates under stated assumptions.

19. Mapping to the Judging Criteria

Criterion	How MIRRORWORLD addresses it
Novelty	Counterfactual simulation + dependency analysis + multi-objective optimization + explainability
Practicality	Targets a real, recurring university planning problem; configurable and uses standard tools
Impact	Quantified reduction in conflicts and resource waste against a documented baseline
Implementation	Working backend engines, database, API and interactive frontend
Presentation	Live five-minute demo where judges change conditions and see recalculated results
SDG alignment	SDG 9, 11, 12 with measurable indicators

20. Compliance with Competition Rules

Rule	Status
Bachelor-level university students only	[Confirm all members]
Maximum three members; each student in one team	[Confirm]
Valid student ID at event	[Bring on 6–7 Nov]
Abstract at registration	Section 2
Project report of at most two A4 pages	Section 24 outline
Safe, hazard-free demonstration	Software only
Original work, not submitted or awarded elsewhere	[Confirm]; cite all libraries and datasets used
No copied or third-party work	Open-source libraries are acknowledged as dependencies; all application logic is the team's own

21. Live Demonstration Script (5 minutes)

1. Show the baseline campus and its KPIs (1,000 students, 20 rooms, 4 labs).
2. Increase students to 1,200 and click **Simulate Future**; show the new capacity and conflict problems.
3. Click **Find Better Plans**; show at least two feasible plans (utilization-first and balanced).
4. Open the comparison view: metrics, trade-offs, Pareto chart and revised timetable.
5. Show the explanation of why the recommended plan is better and what it sacrifices.
6. Let a judge pick a condition (for example, close a lab) and run recovery live; show affected resources in the graph and the new arrangement.
7. Keep a prepared, tested dataset and a recorded backup video in case of network or hardware issues.

22. Limitations and Future Work

- Uses a synthetic dataset unless real campus data is supplied.
- Energy values are estimates; occupancy, weather and air-conditioning behavior are not modeled.
- Student movement uses a simplified distance model.
- Future work: integration with real timetable systems (CSV/API), live IoT energy data, richer sensitivity analysis, multi-campus support, learning-based preference tuning once real data exists.

23. Implementation Plan (time-boxed)

Phase	Work	Output
1	Database, seed data, baseline timetable, metrics	Working campus model
2	Scenario builder, simulation engine, comparison view	What-if simulator
3	CP-SAT optimizer, plans A–D, explanation engine	Decision-support engine
4	Dependency graph, disruption recovery, history, polish, tests, report	Demo-ready build

Suggested roles: Member 1 – optimization and simulation engines; Member 2 – backend API and database; Member 3 – frontend, visualization, report and demo.

If time is short, protect this minimum: campus model, baseline, scenario builder, simulation metrics, optimizer, comparison view and explanations. Treat the copilot, map and sensitivity analysis as optional.

24. Two-Page Project Report Outline

Page 1: Title and abstract; problem statement; proposed solution; technical methodology (data model, constraints, objective, simulation).

Page 2: Implementation and screenshots; actual results table; innovation; SDG contribution and indicators; limitations and future work.

Keep it within two A4 pages, use one architecture figure and one results table, and report only measured results.

25. Registration Checklist

- Team name and members, with student IDs and university details.
- Project title, category and SDGs.
- Abstract (Section 2).
- Two-page report (Section 24).
- Confirmation of originality and eligibility.
- Working demo, backup video, and laptop and charger ready for 6–7 November 2026 at BCFCC, Agargaon, Dhaka.
- Check the official registration deadline and any additional form fields, since they were not included in the details provided.

26. Team Details

Field	Detail
Team name	[Team Name]
University	[University Name]
Department	[Department]
Team contact (email / phone)	[Email / Phone]

Member	Name	Student ID	Department / Batch	Role	Contact
Member 1	[Name]	[ID]	[Dept. / Batch]	Optimization and simulation engines	[Email / Phone]
Member 2	[Name]	[ID]	[Dept. / Batch]	Backend API and database	[Email / Phone]
Member 3	[Name]	[ID]	[Dept. / Batch]	Frontend, visualization, report and demo	[Email / Phone]

All members will carry their valid student ID to the event on 6–7 November 2026.